# 这就是开源

## 第二讲：Git 与 Github

潘宇琦　申鹏威

2026 年 11 月

# 回顾上一讲

## 如何协作？

在上一讲中，我们认识了操作系统的发展历史，什么是开源以及开源的意义。但现在仍然面临一个问题：当我们真正开始接触开源时，如何高效地参与开源项目、管理代码，以及与其他开发者协作？

我在上一讲提到开源之所以发展得如此迅速是因为计算机的普及和互联网的发展，但是仅仅依靠这两点还是不够的。我们需要一个强力的工具，这个工具就是 Git。

# Git 的前世今生

## 最初

最开始开发 Linux 的时候，大家交流代码提交补丁主要是依靠电子邮件，然后林纳斯把通过测试的补丁放到自己维护的代码树中，最终由林纳斯推出整个代码树的新版本，如果想查看新版本做了哪些改进，只能使用 diff 这一个工具，非常的不便。

## 版本控制系统 (Version Control System，VCS)

一种用于记录文件或项目随时间发生的变化，并对这些变化进行管理的工具。

- 记录每一次修改
- 查看不同版本之间的差异
- 恢复到过去的某个版本
- 追踪修改者以及修改原因
- 协调多个开发者共同开发项目

# Git 的前世今生

## Git 诞生

- 2002 年，Linux 内核开发团队开始使用一款名为 BitKeeper 的商业分布式版本控制系统。当时该软件允许 Linux 社区免费使用。
- 2005 年，BitMover 公司与 Linux 内核开发社区发生矛盾，Linux 社区失去了免费使用 BitKeeper 的权利。
- Linus Torvalds 决定自己开发一套版本控制系统，以满足 Linux 内核的大规模协作需求。
- 2005 年 4 月 7 日，Git 的第一个提交诞生。此后，Git 逐渐发展成为今天广泛使用的版本控制系统。

## BitKeeper 后来：

于 2016 年宣布开源，使用 Apache 2.0 许可证，于 2020 年停止开发。

# 如何安装 Git

## 下载安装包

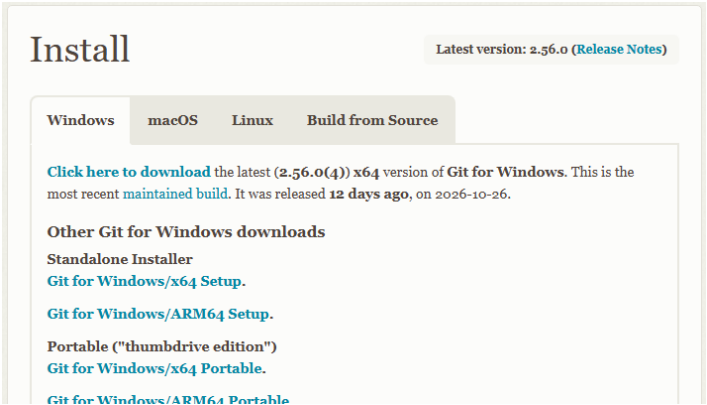
我们先找到 Git 的官网，https://git-scm.com，找到 Latest source release，点击 Install for Windows.



# 如何安装 Git

## 下载安装包

我们能找到很多安装选项：



Install

Latest version: 2.56.0 (Release Notes)

Windows | macOS | Linux | Build from Source

Click here to download the latest (2.56.0(4)) x64 version of Git for Windows. This is the most recent maintained build. It was released 12 days ago, on 2026-10-26.

Other Git for Windows downloads

Standalone Installer

Git for Windows/x64 Setup.

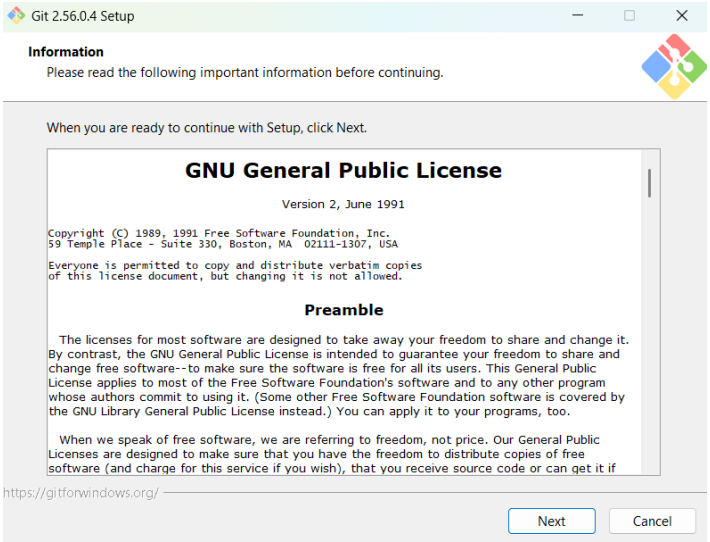
Git for Windows/ARM64 Setup.

Portable ("thumbdrive edition")

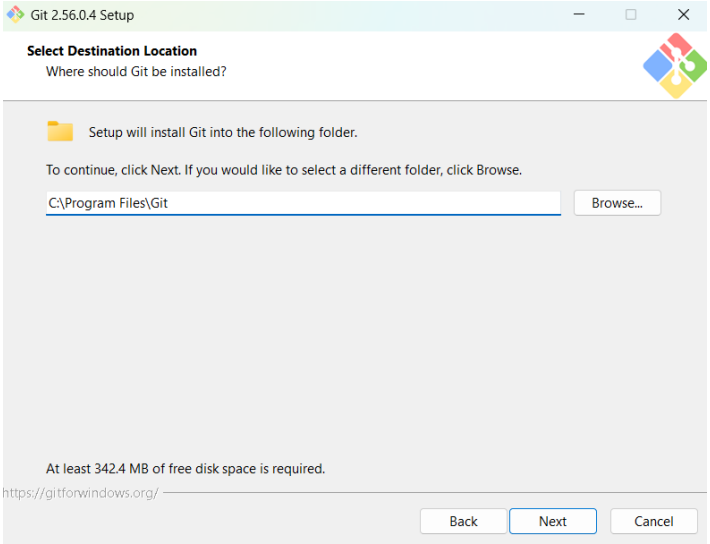
Git for Windows/x64 Portable.

Git for Windows/ARM64 Portable

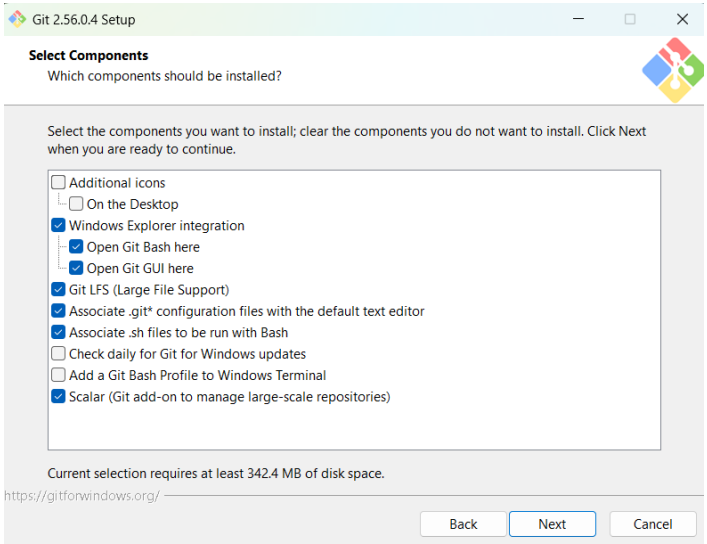
# 如何安装 Git


Git 2.56.0.4 Setup
Information
Please read the following important information before continuing.
When you are ready to continue with Setup, click Next.
GNU General Public License
Version 2, June 1991
Copyright (C) 1989, 1991 Free Software Foundation, Inc.
59 Temple Place - Suite 330, Boston, MA 02111-1307, USA
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.
Preamble
The licenses for most software are designed to take away your freedom to share and change it.
By contrast, the GNU General Public License is intended to guarantee your freedom to share and
change free software--to make sure the software is free for all its users. This General Public
License applies to most of the Free Software Foundation's software and to any other program
whose authors commit to using it. (Some other Free Software Foundation software is covered by
the GNU Library General Public License instead.) You can apply it to your programs, too.
When we speak of free software, we are referring to freedom, not price. Our General Public
Licenses are designed to make sure that you have the freedom to distribute copies of free
software (and charge for this service if you wish), that you receive source code or can get it if
https://gitforwindows.org/
Next
Cancel

# 如何安装 Git

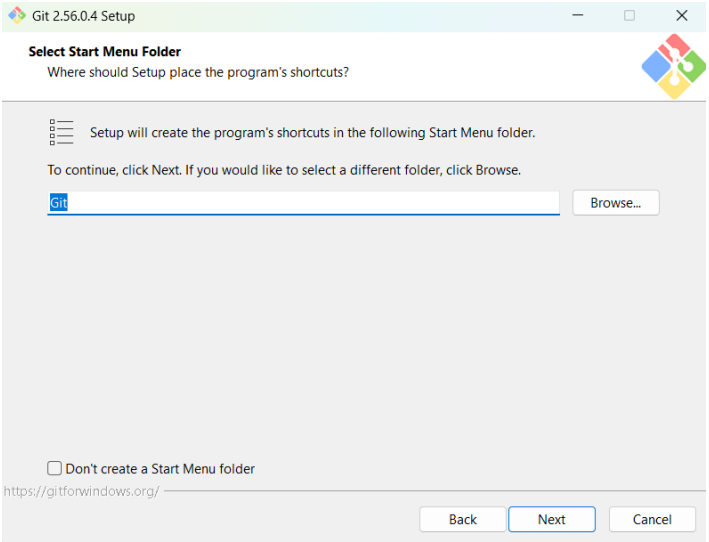

Git 2.56.0.4 Setup
Select Destination Location
Where should Git be installed?
Setup will install Git into the following folder.
To continue, click Next. If you would like to select a different folder, click Browse.
C:\Program Files\Git
Browse...
At least 342.4 MB of free disk space is required.
https://gitforwindows.org/
Back
Next
Cancel

# 如何安装 Git

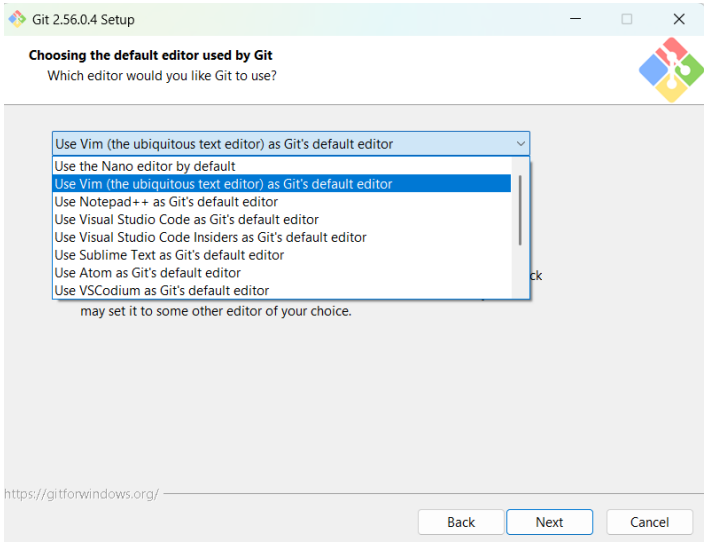



这张图片会和现场演示稍有不同，以现场演示为准。

# 如何安装 Git

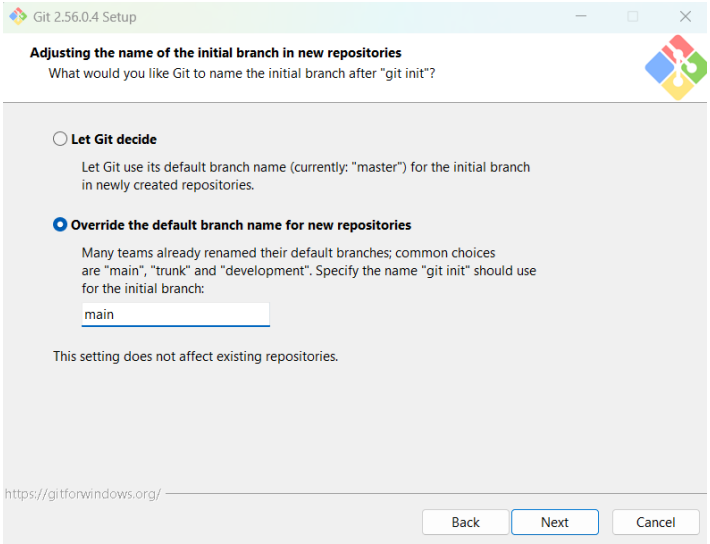

Git 2.56.0.4 Setup
Select Start Menu Folder
Where should Setup place the program's shortcuts?
Setup will create the program's shortcuts in the following Start Menu folder.
To continue, click Next. If you would like to select a different folder, click Browse.
Git
Browse...
Don't create a Start Menu folder
https://gitforwindows.org/
Back
Next
Cancel

# 如何安装 Git

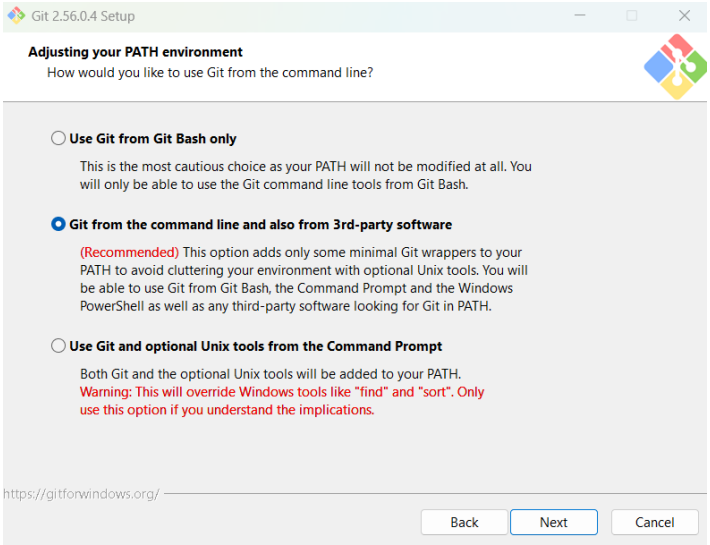



这张图片会和现场演示稍有不同，以现场演示为准。

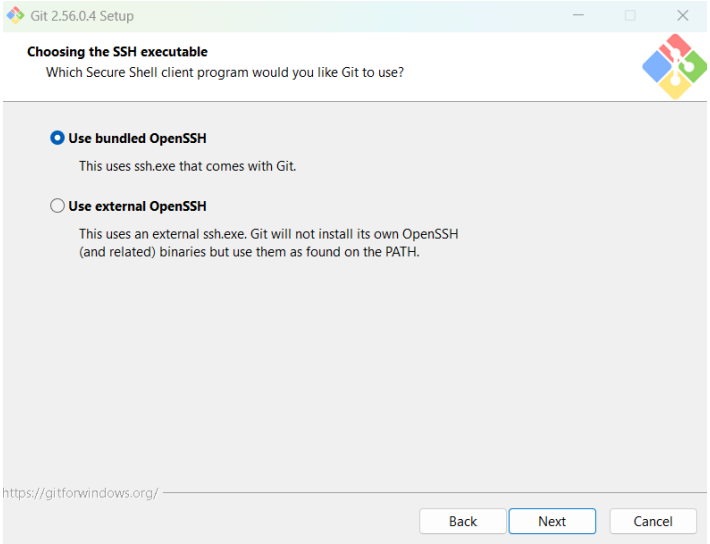
# 如何安装 Git


Git 2.56.0.4 Setup
Adjusting the name of the initial branch in new repositories
What would you like Git to name the initial branch after "git init"?
Let Git decide
Let Git use its default branch name (currently: "master") for the initial branch in newly created repositories.
Override the default branch name for new repositories
Many teams already renamed their default branches; common choices are "main", "trunk" and "development". Specify the name "git init" should use for the initial branch:
main
This setting does not affect existing repositories.
https://gitforwindows.org/
Back
Next
Cancel

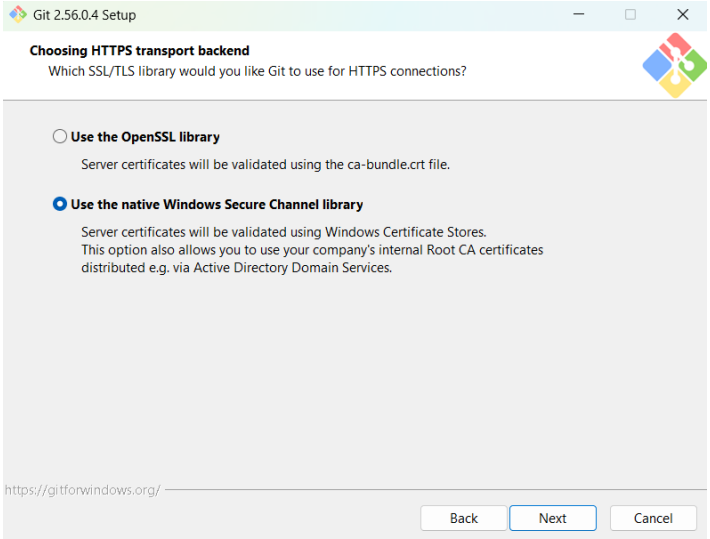
# 如何安装 Git


Git 2.56.0.4 Setup
Adjusting your PATH environment
How would you like to use Git from the command line?
Use Git from Git Bash only
This is the most cautious choice as your PATH will not be modified at all. You will only be able to use the Git command line tools from Git Bash.
Git from the command line and also from 3rd-party software
(Recommended) This option adds only some minimal Git wrappers to your PATH to avoid cluttering your environment with optional Unix tools. You will be able to use Git from Git Bash, the Command Prompt and the Windows PowerShell as well as any third-party software looking for Git in PATH.
Use Git and optional Unix tools from the Command Prompt
Both Git and the optional Unix tools will be added to your PATH.
Warning: This will override Windows tools like "find" and "sort". Only use this option if you understand the implications.
https://gitforwindows.org/
Back
Next
Cancel

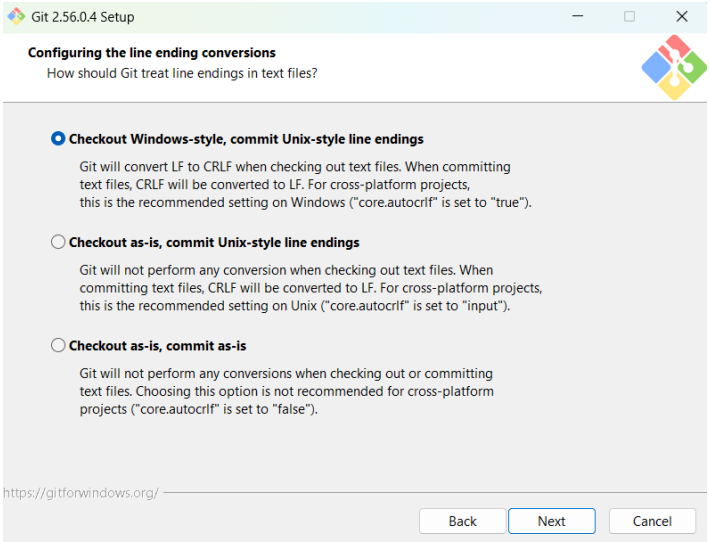
# 如何安装 Git


Git 2.56.0.4 Setup
Choosing the SSH executable
Which Secure Shell client program would you like Git to use?
Use bundled OpenSSH
This uses ssh.exe that comes with Git.
Use external OpenSSH
This uses an external ssh.exe. Git will not install its own OpenSSH
(and related) binaries but use them as found on the PATH.
https://gitforwindows.org/
Back
Next
Cancel

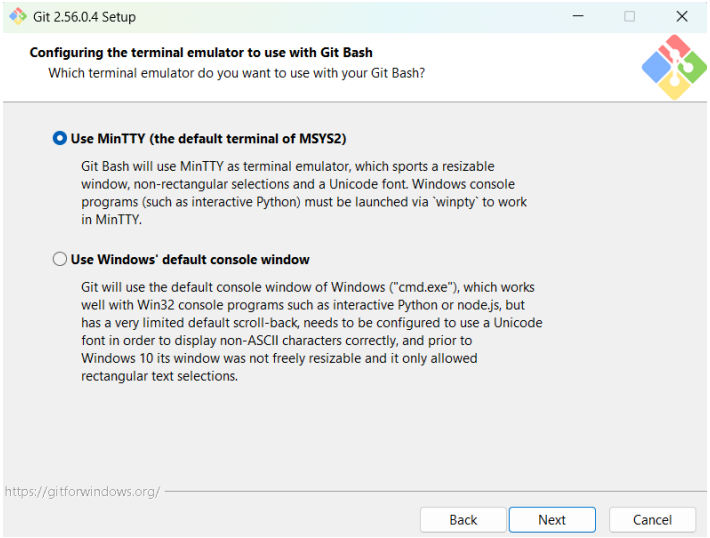
# 如何安装 Git


Git 2.56.0.4 Setup
Choosing HTTPS transport backend
Which SSL/TLS library would you like Git to use for HTTPS connections?
Use the OpenSSL library
Server certificates will be validated using the ca-bundle.crt file.
Use the native Windows Secure Channel library
Server certificates will be validated using Windows Certificate Stores.
This option also allows you to use your company's internal Root CA certificates
distributed e.g. via Active Directory Domain Services.
https://gitforwindows.org/
Back
Next
Cancel

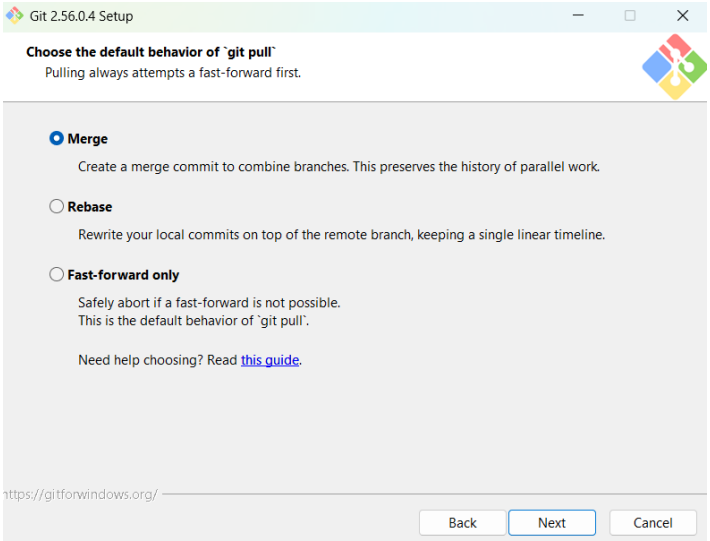
# 如何安装 Git


Git 2.56.0.4 Setup
Configuring the line ending conversions
How should Git treat line endings in text files?
Checkout Windows-style, commit Unix-style line endings
Git will convert LF to CRLF when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects, this is the recommended setting on Windows ("core.autocrlf" is set to "true").
Checkout as-is, commit Unix-style line endings
Git will not perform any conversion when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects, this is the recommended setting on Unix ("core.autocrlf" is set to "input").
Checkout as-is, commit as-is
Git will not perform any conversions when checking out or committing text files. Choosing this option is not recommended for cross-platform projects ("core.autocrlf" is set to "false").
https://gitforwindows.org/
Back
Next
Cancel

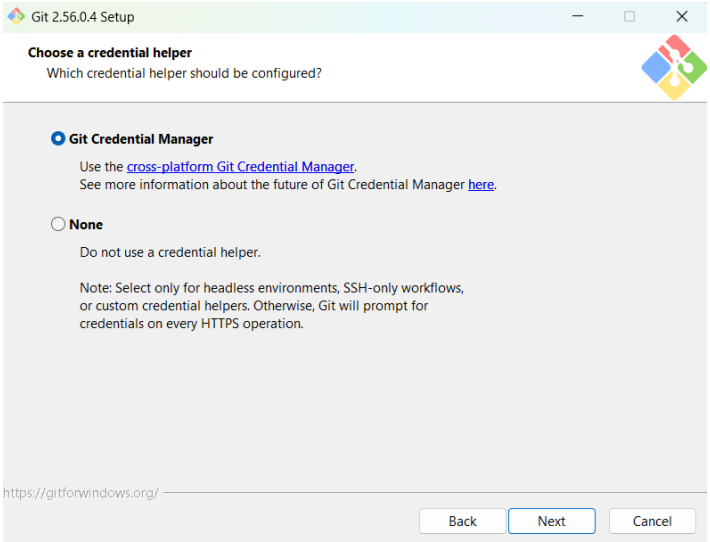
# 如何安装 Git


Git 2.56.0.4 Setup
Configuring the terminal emulator to use with Git Bash
Which terminal emulator do you want to use with your Git Bash?
Use MinTTY (the default terminal of MSYS2)
Git Bash will use MinTTY as terminal emulator, which sports a resizable window, non-rectangular selections and a Unicode font. Windows console programs (such as interactive Python) must be launched via `winpty` to work in MinTTY.
Use Windows' default console window
Git will use the default console window of Windows ("cmd.exe"), which works well with Win32 console programs such as interactive Python or node.js, but has a very limited default scroll-back, needs to be configured to use a Unicode font in order to display non-ASCII characters correctly, and prior to Windows 10 its window was not freely resizable and it only allowed rectangular text selections.
https://gitforwindows.org/
Back
Next
Cancel

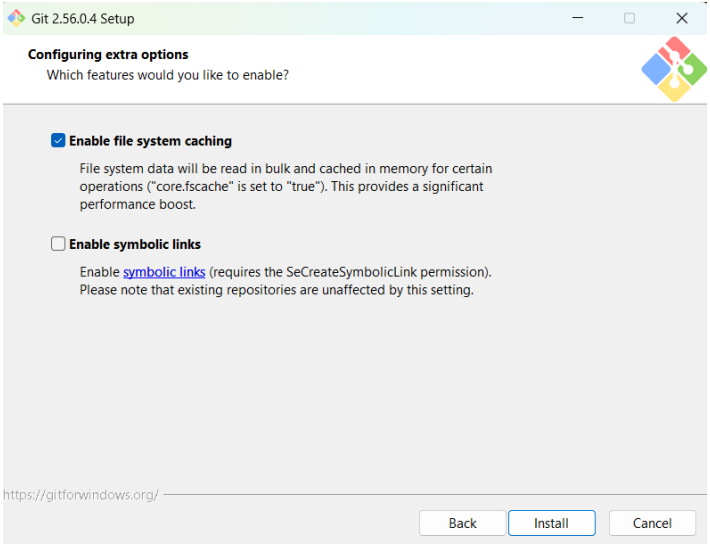
# 如何安装 Git


Git 2.56.0.4 Setup
Choose the default behavior of `git pull`
Pulling always attempts a fast-forward first.
Merge
Create a merge commit to combine branches. This preserves the history of parallel work.
Rebase
Rewrite your local commits on top of the remote branch, keeping a single linear timeline.
Fast-forward only
Safely abort if a fast-forward is not possible.
This is the default behavior of `git pull`.
Need help choosing? Read this guide.
https://gitforwindows.org/
Back
Next
Cancel

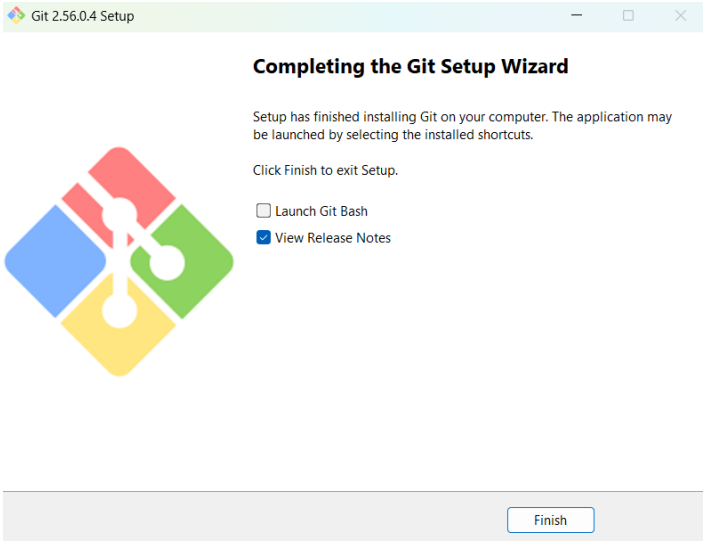
# 如何安装 Git


Git 2.56.0.4 Setup
Choose a credential helper
Which credential helper should be configured?
Git Credential Manager
Use the cross-platform Git Credential Manager.
See more information about the future of Git Credential Manager here.
None
Do not use a credential helper.
Note: Select only for headless environments, SSH-only workflows,
or custom credential helpers. Otherwise, Git will prompt for
credentials on every HTTPS operation.
https://gitforwindows.org/
Back
Next
Cancel

# 如何安装 Git


Git 2.56.0.4 Setup
Configuring extra options
Which features would you like to enable?
Enable file system caching
File system data will be read in bulk and cached in memory for certain operations ("core.fscache" is set to "true"). This provides a significant performance boost.
Enable symbolic links
Enable symbolic links (requires the SeCreateSymbolicLink permission). Please note that existing repositories are unaffected by this setting.
https://gitforwindows.org/
Back
Install
Cancel

# 如何安装 Git




这张图片会和现场演示稍有不同，以现场演示为准。

# Git 安装成功

## 安装成功

使用 git –version 命令来判断是否安装成功

```
13793@ForOpenSource UCRT64 ~
$ git --version
git version 2.56.0.windows.4
```

# Git 配置

## 配置用户名和邮箱

使用如下命令来进行配置

- git config –global user.name " 你的用户名"
- git config –global user.email " 你的邮箱"

```
13793@ForOpenSource UCRT64 ~
$ git config --global user.name "Leon"

13793@ForOpenSource UCRT64 ~
$ git config --global user.email "13793425428@163.com"

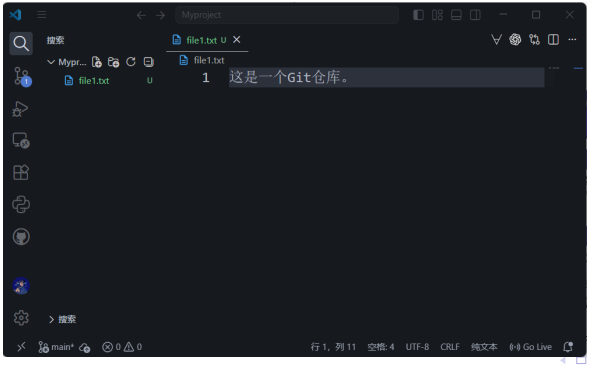
13793@ForOpenSource UCRT64 ~
$ git config --global --list
user.name=Leon
user.email=13793425428@163.com
```

# Git 的基本工作原理

## 仓库

当我们在一个项目文件夹中执行 ‘git init‘ 命令时，Git 会在该文件夹下创建一个名为 ‘.git‘ 的隐藏目录，用于保存版本控制所需的信息。此时，该文件夹就成为了一个 Git 仓库。

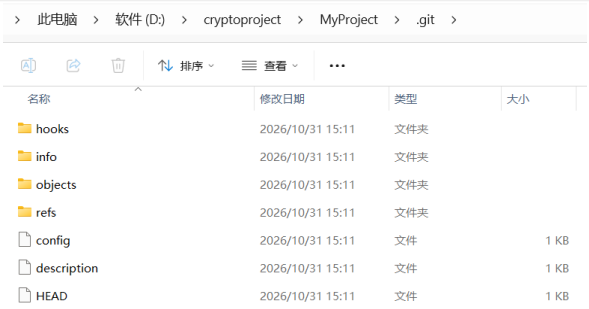
需要注意的是，Git 并不要求初始化的文件夹必须为空，原有文件也不会受到影响。



# Git 的基本工作原理

## .git 文件夹

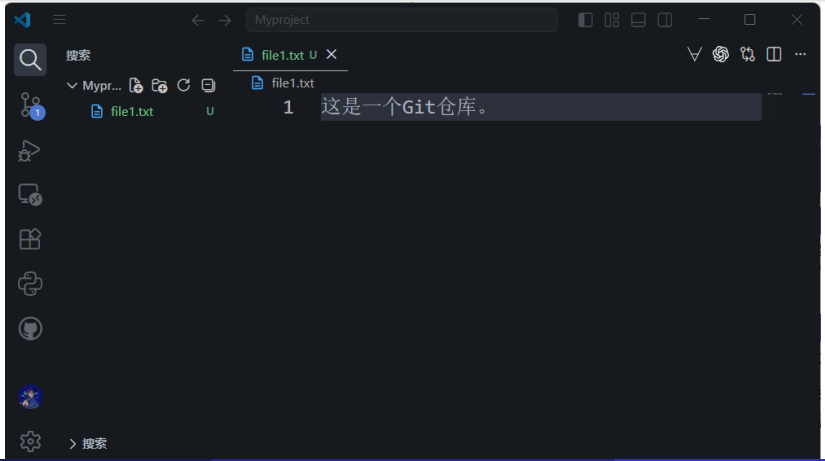
这个文件夹在被初始化为仓库之后，就会增加一个名为.git 的隐藏文件夹，其中会含有版本控制的重要信息。



# Git 的基本工作原理

## 工作区

我们实际看到和编辑的项目文件。对文件进行修改、创建和删除，都是在这里完成的。



# Git 的基本工作原理

## 暂存区

我们使用 git add 命令，将指定文件的新增、修改或删除等变化加入暂存区。暂存区并不是一个需要我们手动管理的普通文件夹，其状态信息主要记录在.git/index 文件中。

```
panyu@LeonHP MINGW64 /d/cryptoproject/MyProject (main)
$ git add file1.txt

panyu@LeonHP MINGW64 /d/cryptoproject/MyProject (main)
$
```

# Git 的基本工作原理

## 版本库

当我们执行 git commit 命令时，Git 会根据暂存区中的内容创建一次新的提交（Commit），记录项目在该时刻的状态，并将相应的版本信息保存在.git 目录中。

每一次提交都有一个唯一的标识符，我们可以使用 git log 命令查看过去的提交记录。

```
panyu@LeonHP MINGW64 /d/cryptoproject/MyProject (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file1.txt


panyu@LeonHP MINGW64 /d/cryptoproject/MyProject (main)
$ git commit -m"Initial commit"
[main (root-commit) d037208] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 file1.txt

panyu@LeonHP MINGW64 /d/cryptoproject/MyProject (main)
$ git log
commit d037208d77d9be008d60b04baf3afeb524474c0e (HEAD -> main)
Author: Leon <panyuqia@163.com>
Date:   Tue Nov 3 23:00:13 2026 +0800

    Initial commit
```

# Git 的基本工作原理

```
panyu@LeonHP MINGW64 /d/cryptoproject/MyProject (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file1.txt


panyu@LeonHP MINGW64 /d/cryptoproject/MyProject (main)
$ git commit -m"Initial commit"
[main (root-commit) d037208] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 file1.txt

panyu@LeonHP MINGW64 /d/cryptoproject/MyProject (main)
$ git log
commit d037208d77d9be008d60b04baf3afeb524474c0e (HEAD -> main)
Author: Leon <panyuqia@163.com>
Date:   Tue Nov 3 23:00:13 2026 +0800

    Initial commit
```

当 PYQ 看到这一页 PPT 时就该开始现场演示了。

# Git 的基本操作